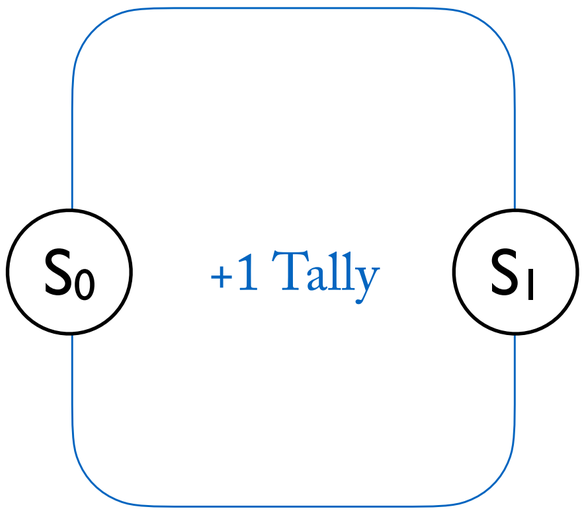
# 30. Turing Machine Abstraction and Composition

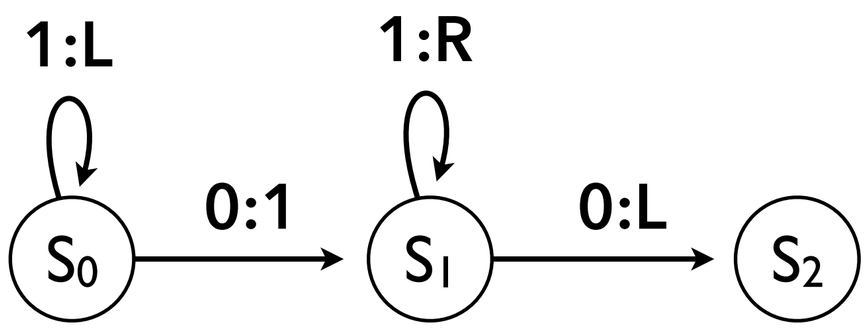
## Functional Abstraction for Subroutines

We can apply functional abstraction to TM's which compute specific functions.

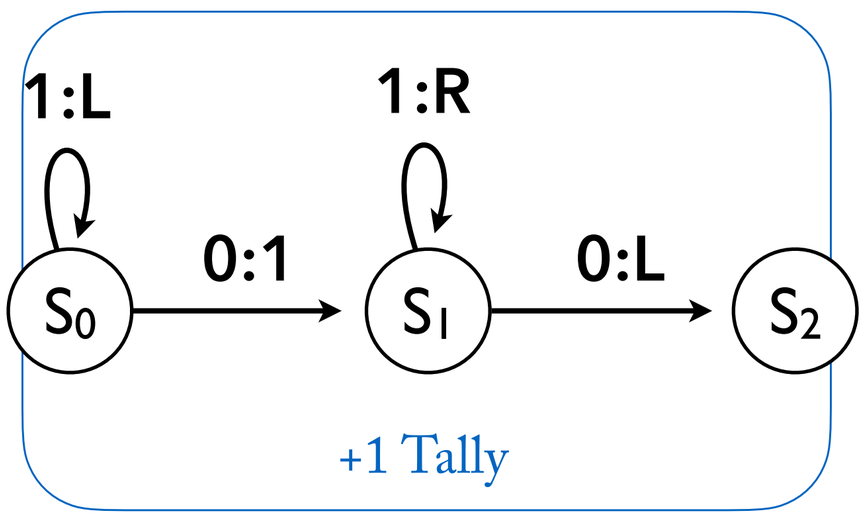



This boxed TM starts in $S_0$ and halts in $S_1$. By the time it halts, it will have computed +1 T.

Boxed TMs will be easier to combine into larger machines if they start *and* finish in standard position. For example, this TM computes +1 T and returns to standard position:




We can box this machine it by drawing a frame around it that intersects only with initial state and the halting state:




Then we can kick away the internal states:

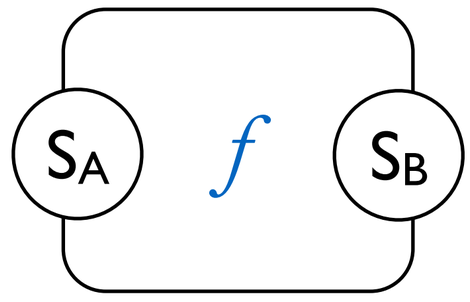


## Constraints on Boxing

To prevent technical errors, we have to impose a few constraints on when a boxed machine may be introduced.

A machine M can be **boxed** only when:

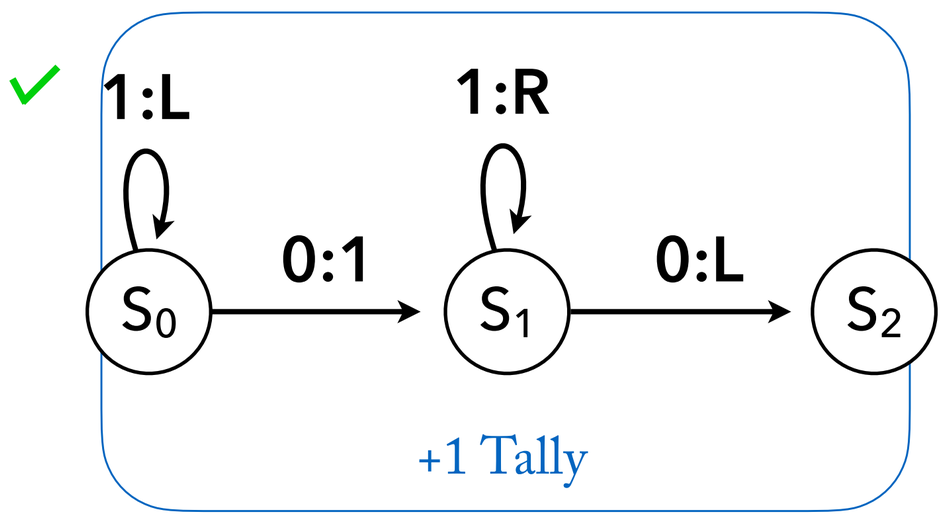
1. M computes a function *f*.
2. $S_A$ is the **initial state** of the machine.
3. $S_B$ is the only **terminal node** of the machine:
   a) M halts in standard position;
   b) M halts in $S_B$;
   c) M halts in no other state (given the task);
   d) there are no transitions from $S_B$ to other internal states of M.



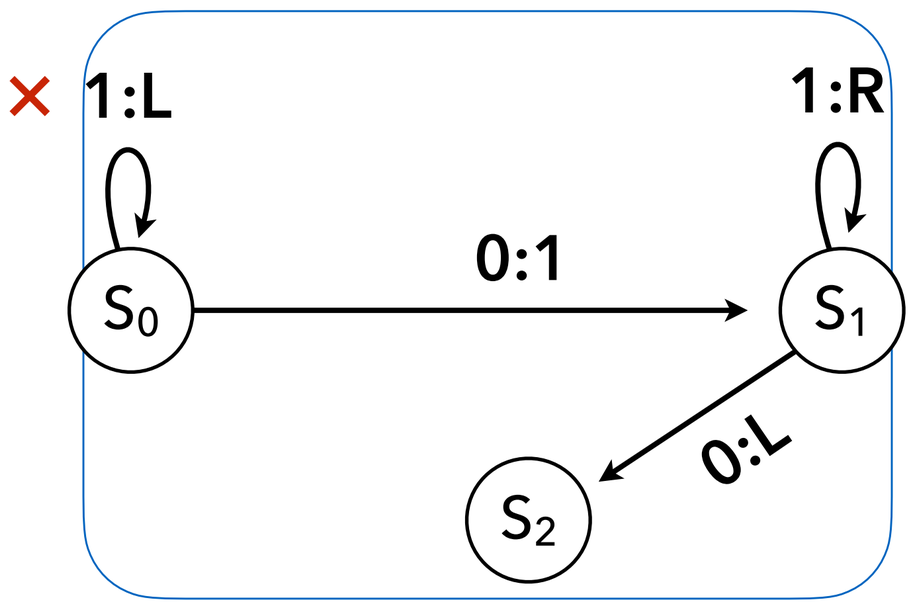

A boxed machine M can only be used in a larger circuit when there are no transitions from $S_A$ to any states outside of M.

---

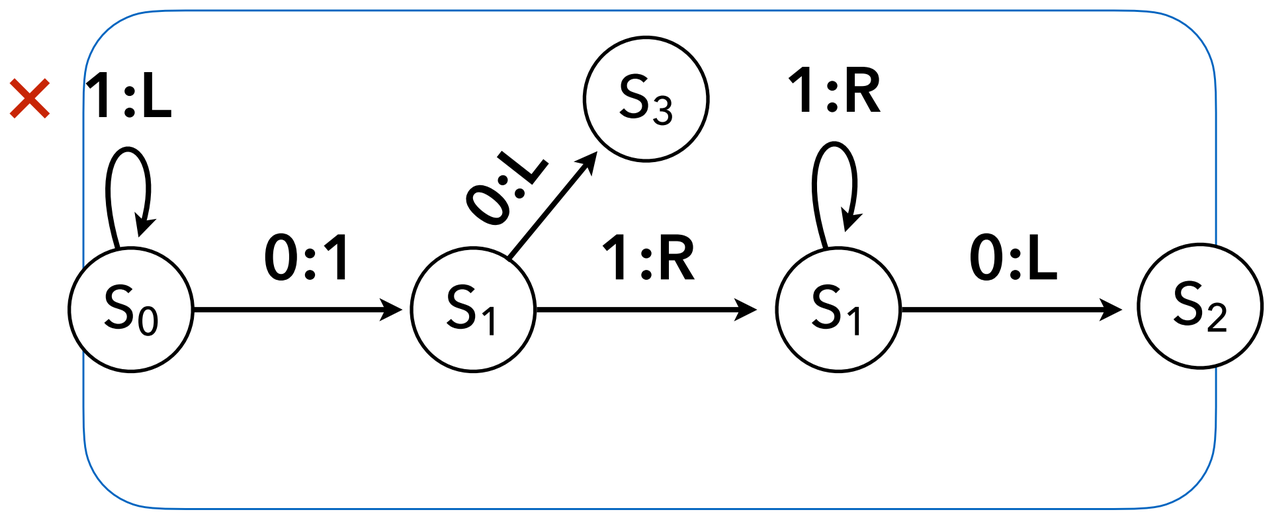
So this is right:




But this is wrong:

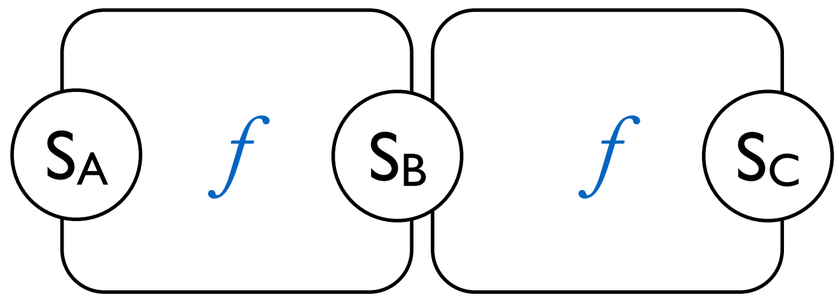



And this is wrong:

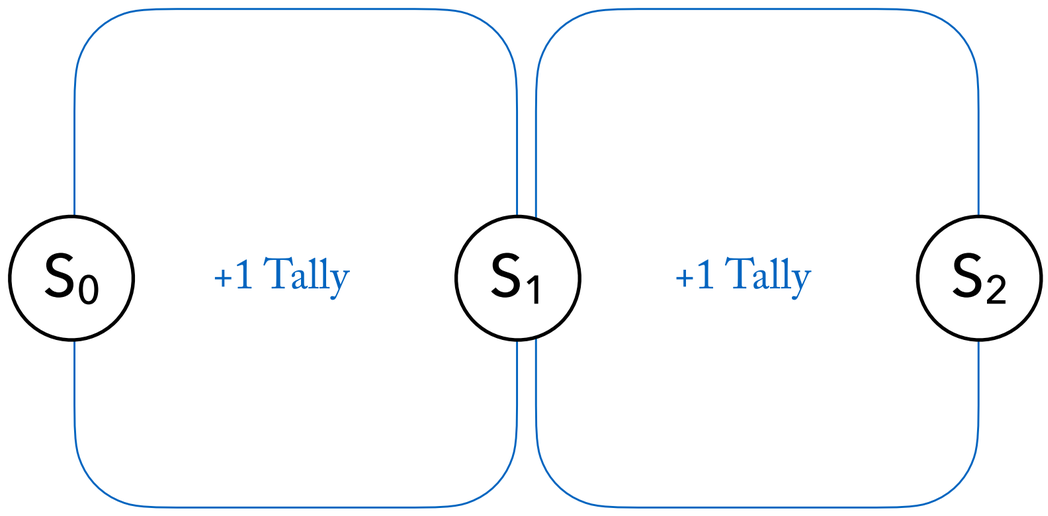


# Function Composition

Boxed machines can be composed together to form larger machines:




For example, this machine computes +2 T.




---

Boxed machines can be integrated with unboxed elements as well.

This machines moves left over a first block and then performs +1 T on a second block, then returns to standard position.

